MRGYM – Technical Report

Martim Alves (1240690), Ricardo Monteiro (1240842)

Abstract. This report serves as a guide to understanding the work put into the development of the application MRGym. All work processes will be shown throughout the document with diagrams, code, tests, and reasoning of the choices we made.

Keywords: MRGym, Diagrams, Code, Java, IntelliJ, Model, Tests, Rest API, RestDB, Wger, Architecture, Hardware, Application, Android.

1 Introduction

In the context of the project proposed in the curricular unity of Developing Software and Mobile Systems (DSSMV), we proposed the idea of a gym app, with the means to solve the problem of a trainer not having a source of information for how an exercise is done, and what part it trains.

With the main objective of the project being communication with the world, that through a rest API, we decided to use two APIs, those being Restdb, and Wger. Restdb serves as a database, where the app will store data (eg. Users; workouts; photos), and Wger allows the application to get a variety of Gym exercises, their images, and save workouts. The app also requires the use of android hardware, and for that we decided to use the camera, for tracking progress.

The app helps the user with tracking the progress of their workouts, which they can create and customize with different exercises, with different weights, repetitions, and sets. MRGym also serves as a helper for the user to see their progress throughout their fitness journey with a journal, where they can post photos of themselves, and note their weight at the time.

With the intuition of having different users and keeping information private for each user, there is a register and login system in the system, which saves and coordinates every saved aspect to the correct account. We felt the need to add this system to come closer to what would be a real utilization of the app, where different people would install and use it.

The development of our work has been documented in UML, to help with the understanding of our though process and how the program works. That means diagrams like domain model, class diagram, use cases, SSDs, SDs, and other diagrams will appear throughout the document.

2 Analysis

To understand and solve the problem, first we analyzed how we should build the app. That implies a set of use cases, functional and non-functional requirements, as well as the domain model, a class diagram, a glossary, sequence diagrams, system sequence diagrams, and more.

2.1 Domain Model

The Domain Model represents the main elements of the system graphically, as well as their relationship. To further understand how each element interacts with each other, we first defined the entities and their relationship, allowing a good foundation for the rest of the modeling of the app.

Our domain model, presented in Figure 1, consists of 5 main entities, those being User, WorkoutLog, JournalEntry, WorkoutPlan and Exercise. A user can create any number of workout plans, and those workout plans contain at least one exercise. The user also records up to any number of journal entries.

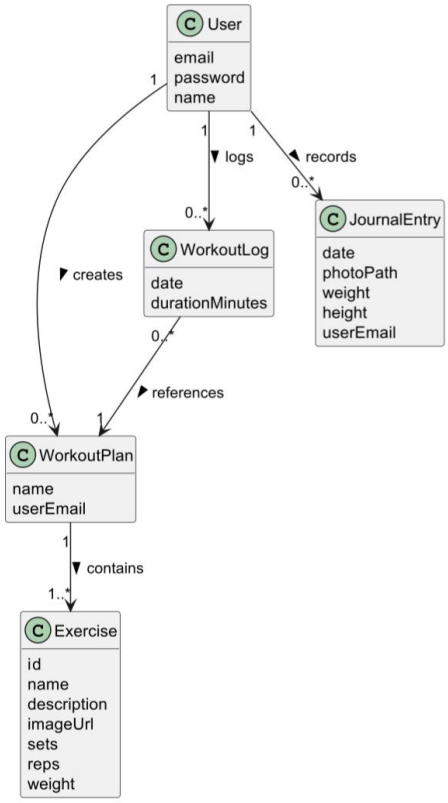


Figure 1

2.2 Glossary

The glossary defines the elements, so that there is a clear understanding of what each element means and represents. Our glossary is demonstrated in Table 1.

Table 1

Technical Term	Definition
Exercise	Planned, structured, and repetitive physical activity designed to improve and maintain physical fitness.
JournalEntry	A photo with details of how much a user weights and how tall a user is.
User	Person who interacts with the app, to track their progress.
Workout Log	Date and duration of a workout.
Workout Plan	Structured personalization of a set of different exercises.

2.3 Functional Requirements

Use cases (UCs) define the functionalities that the user will be able to use during the utilization of the app. In Figure 2 we present our use case diagram, consisting of 11 different use cases, meaning the user will have a variety of options to choose from in our app.

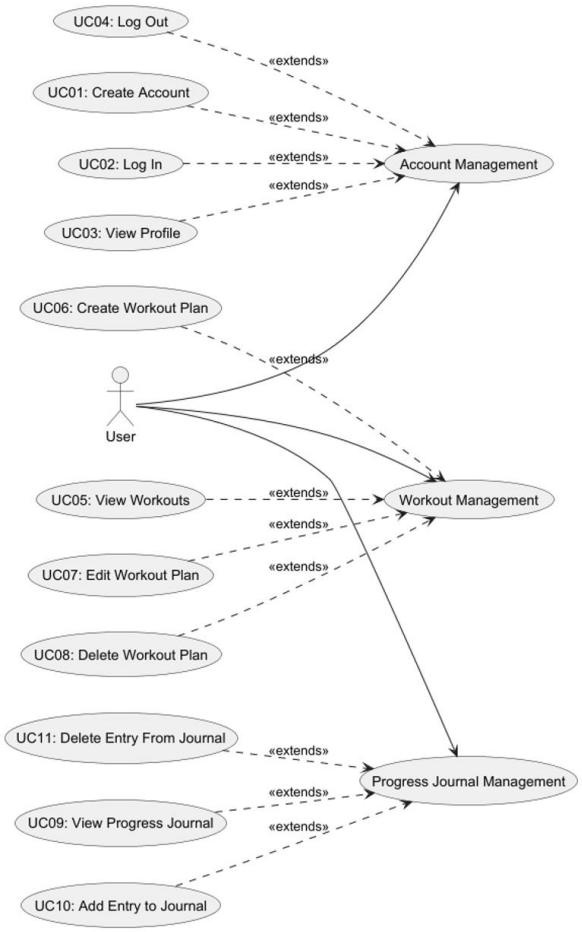


Figure 2

After defining the requirements, the next step is to draw the SSDs (System Sequence Diagrams). These diagrams are a general representation of a functionality, meaning there isn't really any code involved in it yet, but it helps with understanding the system logic, and later build the system diagrams, which will be more precise.

SDD – UC1: Create Account

The user will be able to register an account, as seen in Figure 3. By entering a valid username, e-mail and password, the user will be able to successfully create their account. Accounts are stored in the RestDB API.

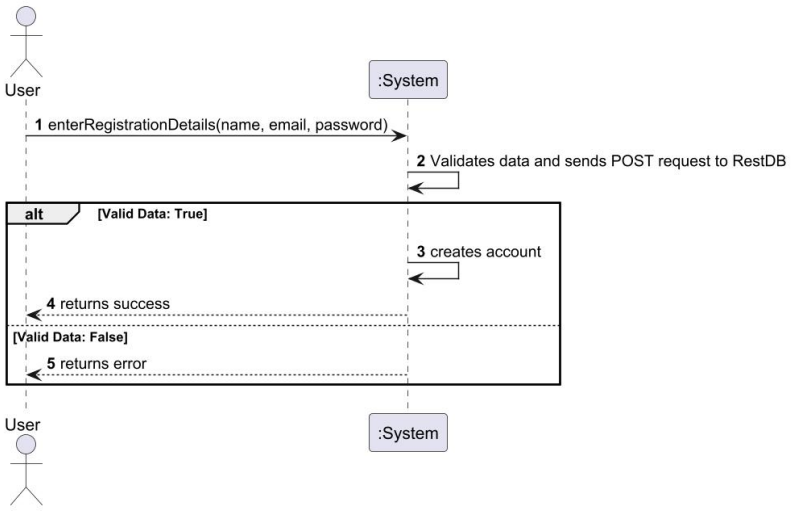


Figure 3

SDD – UC2: Log In

As shown in Figure 4, with the account created, there is also a log in system, meaning that in our application, to access and use the remaining functionalities, the user must have an account created, and must also be logged in to their account.

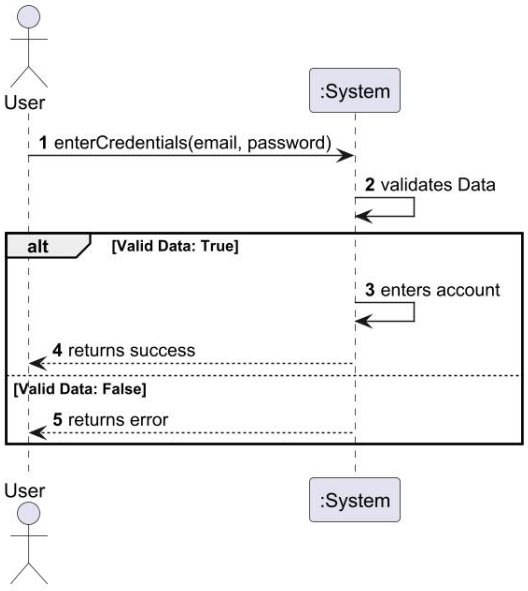


Figure 4

SDD – UC3: View Profile

There is the possibility of viewing the profile that's being used through this use case (present in Figure 5), which makes it simple and intuitive.

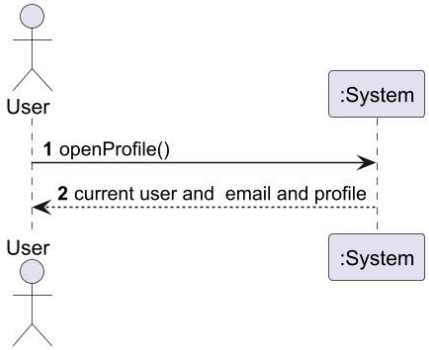


Figure 5

SDD – UC4: Log Out

With having multiple accounts, the log out functionality (shown in Figure 6) is a must in our application.

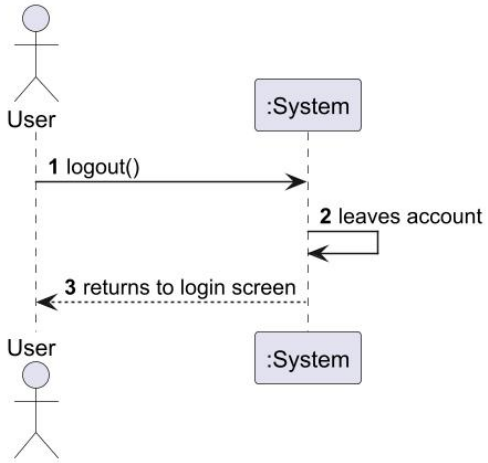


Figure 6

SDD – UC5: View Workouts

The user has the possibility of viewing created workouts, which are unique to each user. The view workout system diagram is shown in Figure 7.

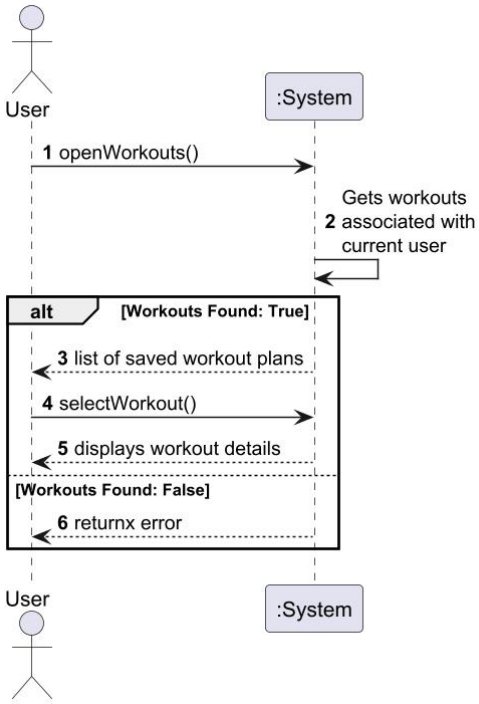


Figure 7

SDD – UC6: Create Workout Plan

Workout plans are created through the system flow shown in Figure 8. This is one of the main functionalities of the program, as it uses the Wger API to GET the various exercises, description and images to display to the user. The workout association with the account they were created in are saved in RestDB.

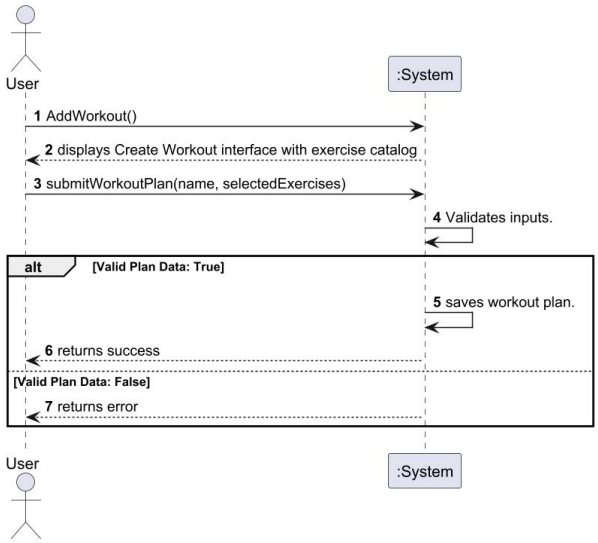


Figure 8

SDD – UC7: Edit Workout Plan

The user can edit workouts (Figure 9), by changing weight, number of repetitions, and number of sets.

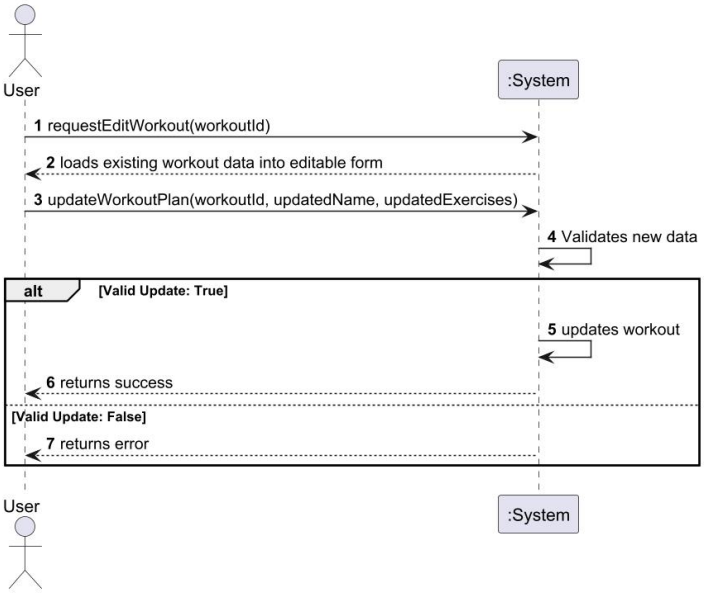


Figure 9

SDD – UC8: Delete Workout Plan
If the user dislikes or simply wants to remove the workout plan they created, they can just delete the workout plan, which will follow the system flow presented in Figure 10.

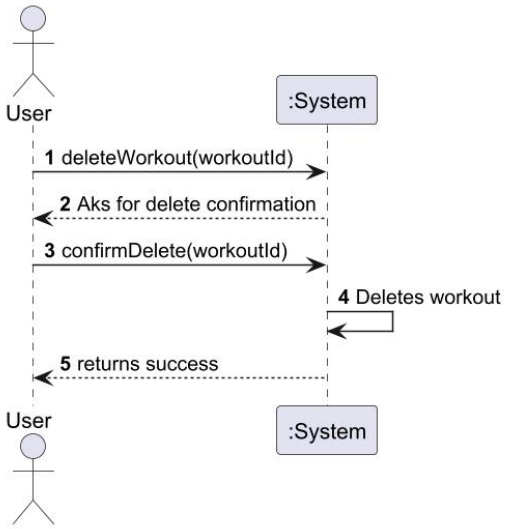


Figure 10

SDD – UC9: View Progress Journal
The other main functionality in our application is the progress journal. In Figure 11 there is a simple description of how it works.

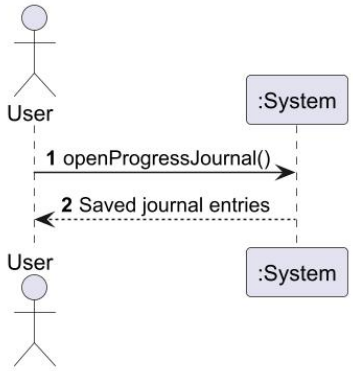


Figure 11

SDD – UC10: Add Entry to Journal
Although the progress journal appears empty at first, there is the functionality for the user to add photos, track weight and height, through their camera and data (Figure 12).

First the app asks for camera permission, so that it ensures there is no crucial crash during the flow of the functionality. Then using the RestDB API, it will save the photos and data, as well as in which account it was saved it, so it doesn't appear for every user.

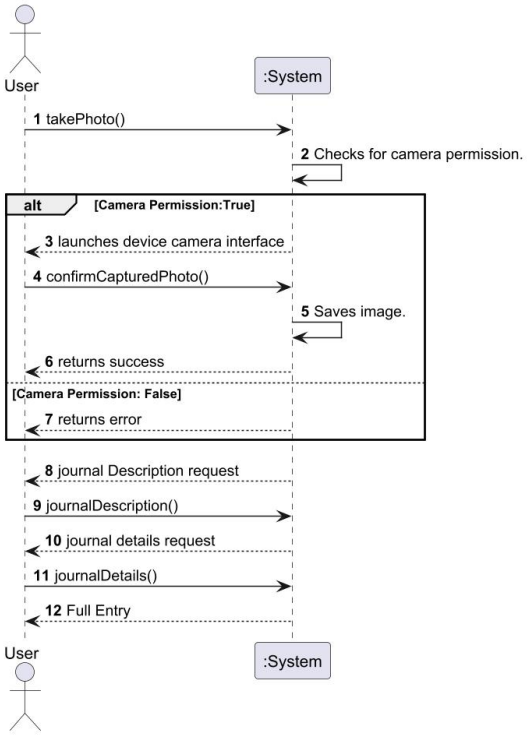


Figure 12

SDD – UC11: Delete Entry from Journal

Just like in workouts, there is the possibility to delete journal entries, shown in the system flow in Figure 13.

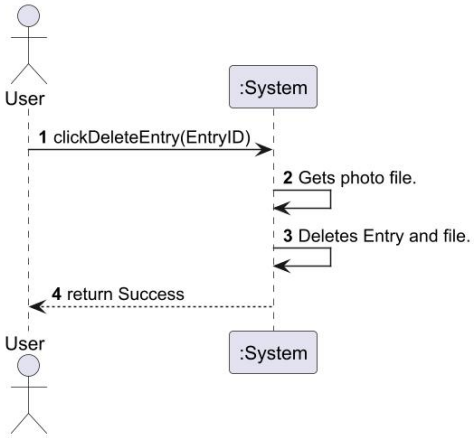


Figure 13

After drawing the SSDs, our next step was to build the SD's, which as referenced before, are a more detailed and complexed view of how the code will function and classes will interact.

SD – UC1: Create Account
In Figure 14 there is a more detailed description of how the code will run during the creation of an account.

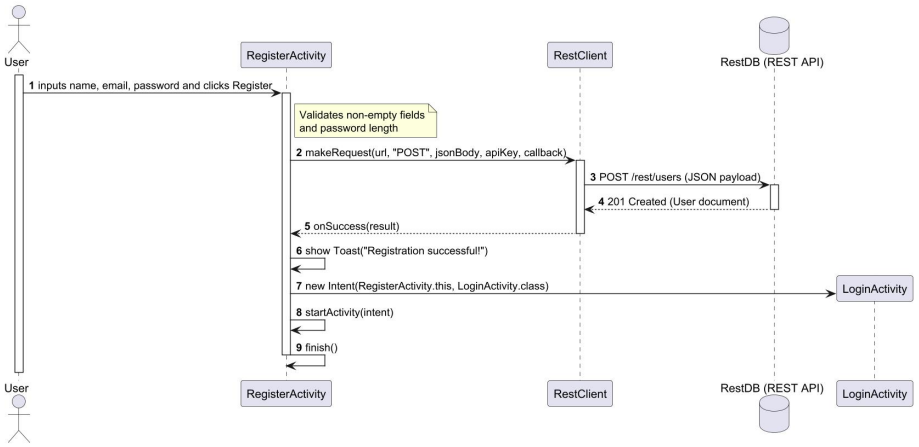


Figure 14

SD – UC2: Log In
Logging in will have the structural flow shown in Figure 15.

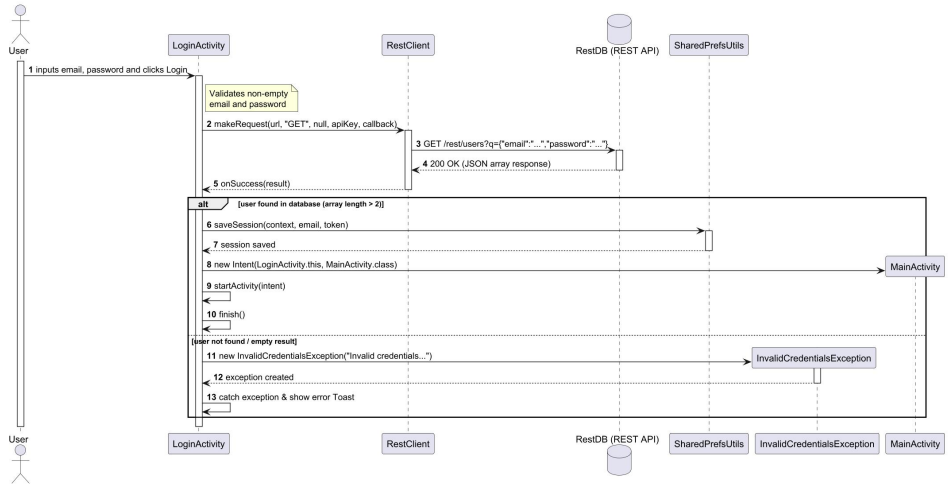


Figure 15

SD – UC3: View Profile
Seeing the current profile, will be run with the steps in Figure 16.

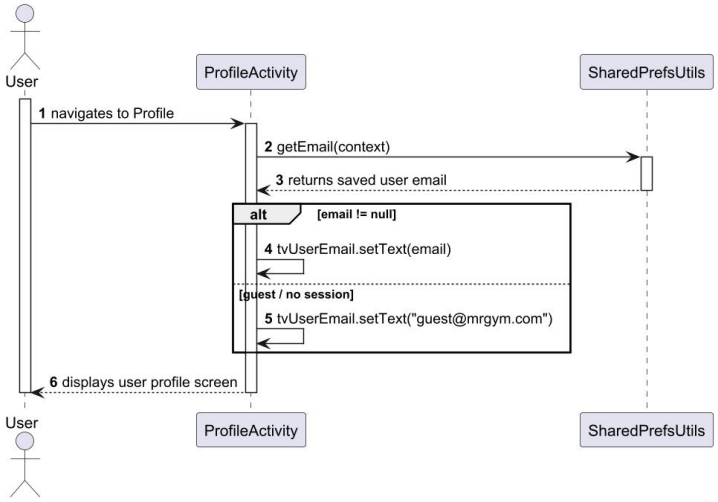


Figure 16

SD – UC4: Log Out
Figure 17 shows how the user can logout, and how the system clears the user session, returning to the log in interface.

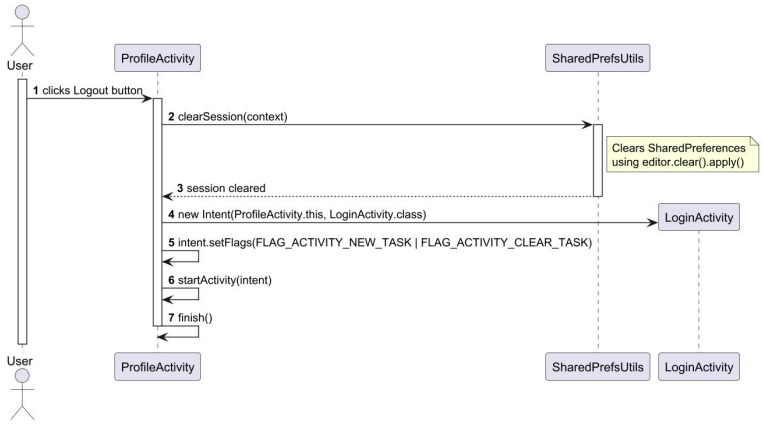


Figure 17

SD – UC5: View Workouts

In Figure 18, there's presented the diagram that allows the viewing of saved workouts, managed by RestDB. By getting the workouts and users, it is able to properly show the user's saved workouts.

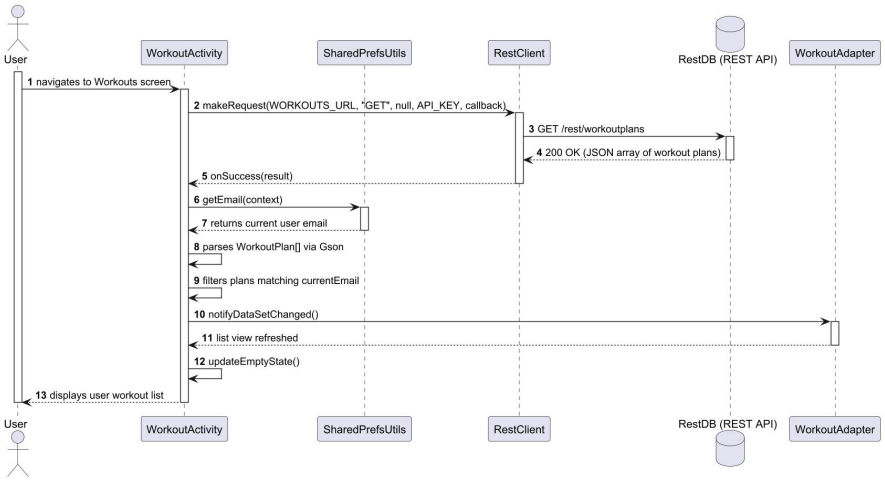


Figure 18

SD – UC6: Create Workout Plan

To save workouts, both RestDB and Wger APIs have a crucial function in both showing and saving the correct information of the workouts, as seen in Figure 19.

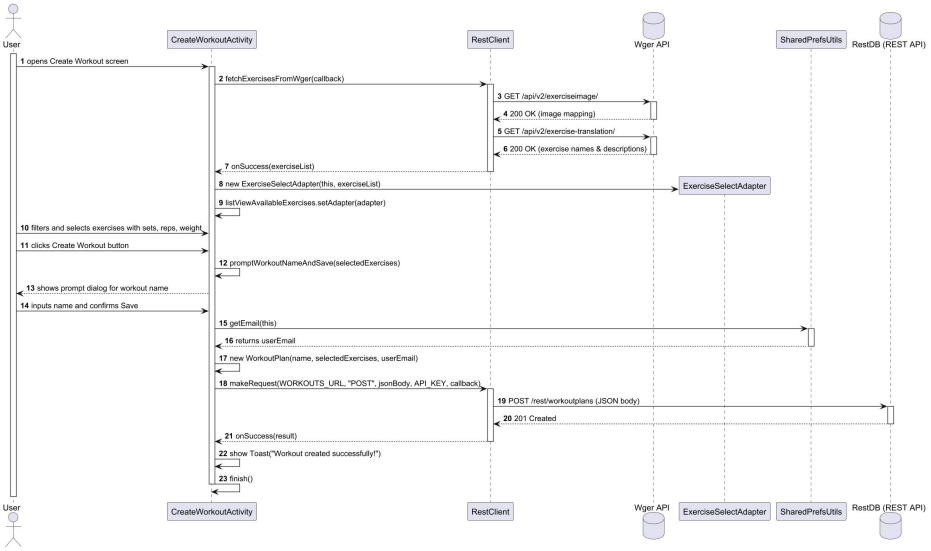


Figure 19

SD – UC7: Edit Workout Plan

The user can edit the workout through the flow shown in Figure 20.

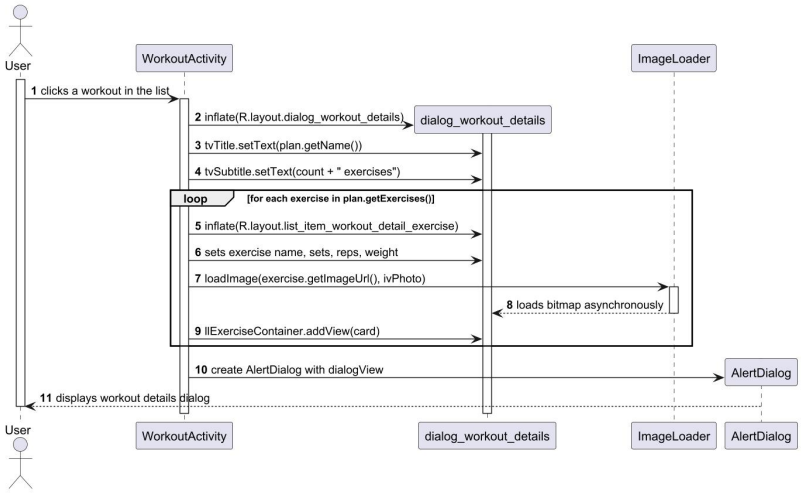


Figure 20

SD – UC8: Delete Workout Plan

To delete a workout from the system database, RestDB API is used once again (Figure 21).

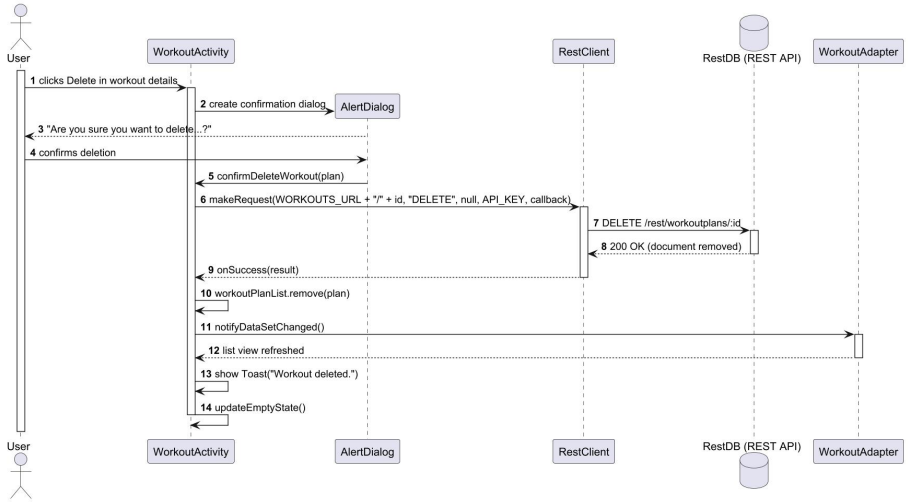


Figure 21

SD – UC9: View Progress Journal

Figure 22 shows how the user can see saved progress journal entries.

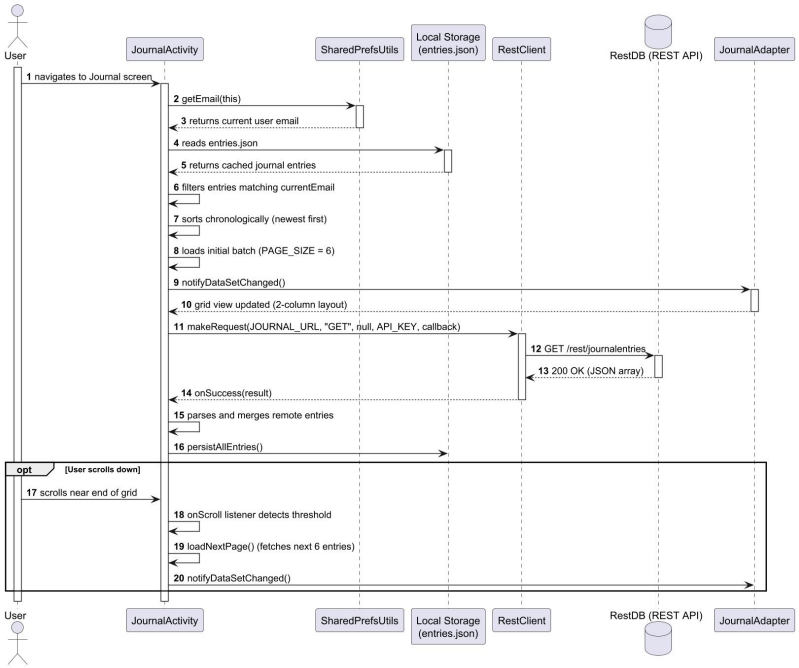


Figure 22

SD – UC10: Add Entry to Journal
Figure 23 shows how the user can save a journal entry. This is done by using the RestDB API once more.

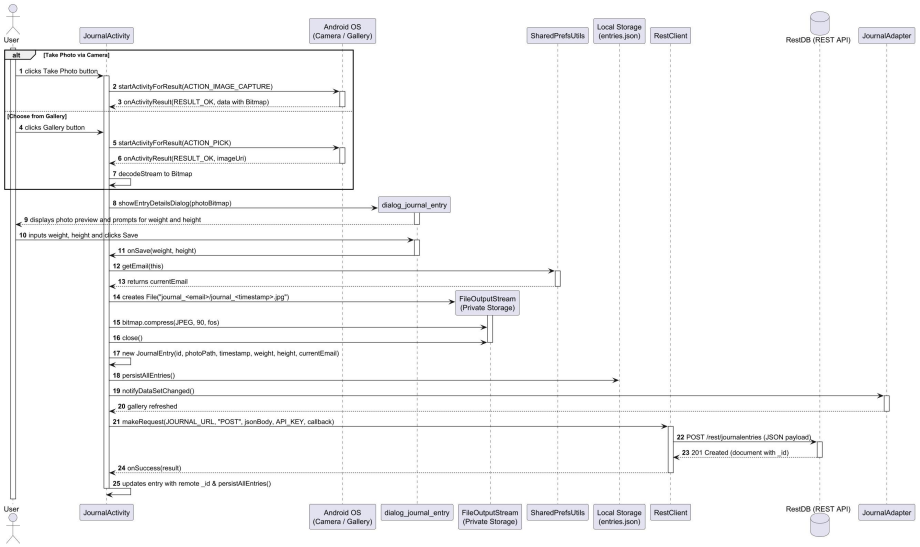


Figure 23

SDD – UC11: Delete Entry from Journal
Finally, using the flowchart in Figure 24, the program will be able to delete the journal entry from the Rest API.

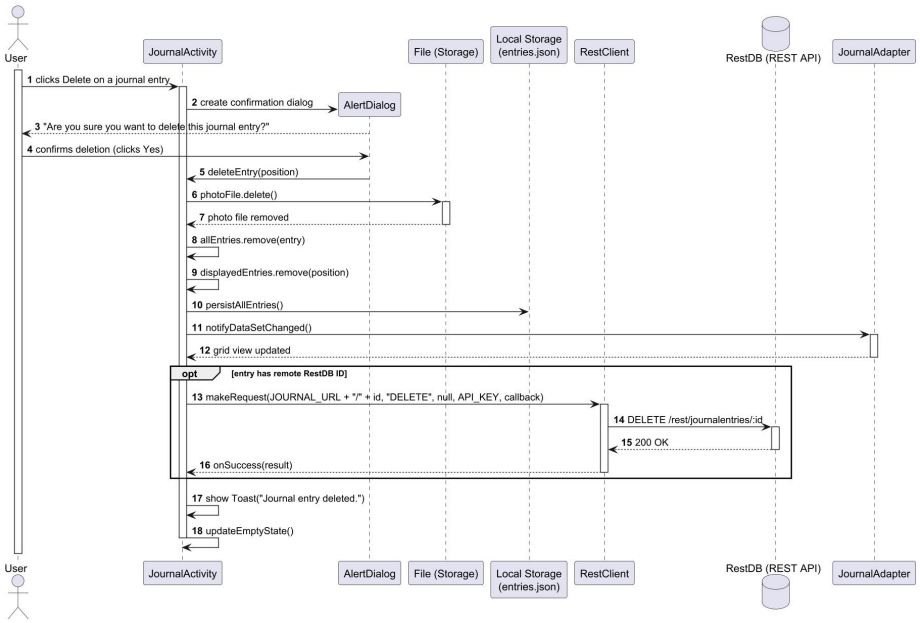


Figure 24

2.4 Non-Functional Requirements

Usability

- Simple and intuitive interface
- Profile system for different users

Reliability

- The app should handle errors without crashing.
- The app should waste little phone charge.
- User data is stored and retrieved consistently.

Performance

- Screens should load within a reasonable amount of time.
- The app should run consistently with good performance.

Supportability

- Code is well structured.
- Tests should be performed and successful.

+

Design Constraints

- The application must follow the android application design conventions.

Implementation Constraints

- The application should be built using Android Studio or IntelliJ IDEA with Android development support.

- The application must have network connection to function properly.
- The application must interact with the world, through rest APIs.
- The application must use the android hardware.

Interface Constraints

- The application must provide an interface compatible with Android devices.

3 System architecture

The system architecture describes the general structure of the app. By presenting the system's physical architecture (Figure 25), the code organization (Figure 26), the model class diagram (Figure 27), the views class diagram (Figure 28), the controller class diagram (Figure 29), the external components class diagram (Figure 30) and the user interface (Figure 31 through Figure 38), there is a clear visualization of how the application will run, and how it will look.

3.1 Physical Architecture

The Physical Architecture allows to visualize how the app will run on the device, and how it connects to the outside world. As seen in Figure 25, Network allows the connection between MRGym, and the Rest APIs used for it to properly function. Without an internet connection, the app will run, but all saved workouts and journal entries won't appear (although they are still saved in RestDB). This also means the app won't be able to retrieve exercise information from the Wger API, making it so that this service will not be displayed.

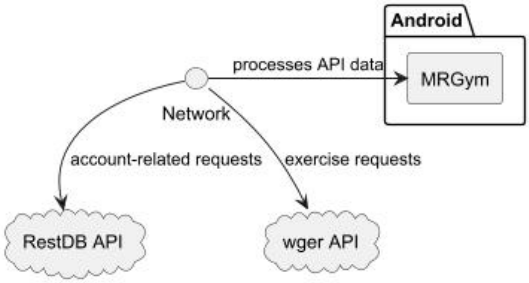


Figure 25

3.3 Code Organization

In this project, we used the MVC (Model, View, Controller) approach, also with different packages (such as exceptions, network and utils). While Model handles the system's data and operations and Views handle the showing and collecting of data to the user, exceptions handle operations that should not occur so that if they do, the system doesn't crash. Utils contains reusable helper classes that provide functionality throughout the application. The network package allows communication with external APIs and is responsible for making HTTP requests and retrieving information. Controller acts as a bridge between all, getting functions from every package, so that the application runs as desired.

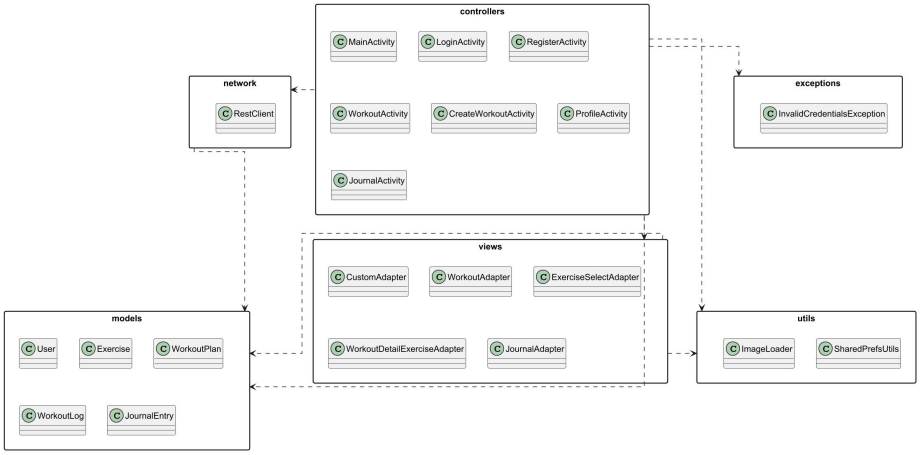


Figure 26

3.4 Model Class Diagram

The model class diagram represents the data structure of the application. It shows the different model classes used by the app, their attributes, methods and relationships. These classes represent the main entities handled by the application, such as users, exercises, workout plans, workout logs and journal entries.

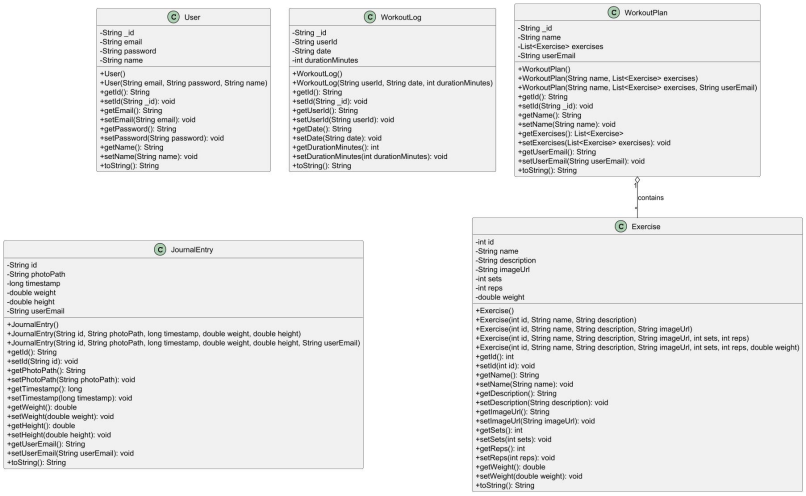


Figure 27

3.4 Views Class Diagram

The views class diagram represents the classes responsible for showing and retrieving information to the user.



Figure 28

3.5 Controller Class Diagram

The controller class diagram illustrates the activities responsible for handling user interactions and controlling the application's flow. It shows the different screens available and how users can navigate them. Each controller is responsible for managing the actions and operations associated with its respective screen.

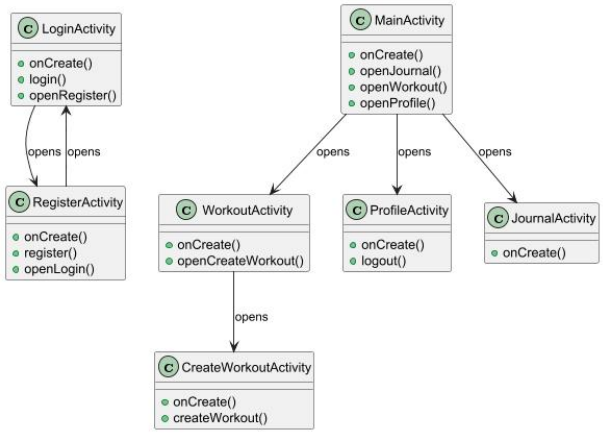


Figure 29

3.6 External Components Diagram

The external components diagram represents the other packages who are used in the application but are not as extensive as the previous ones to be put in their own categories.

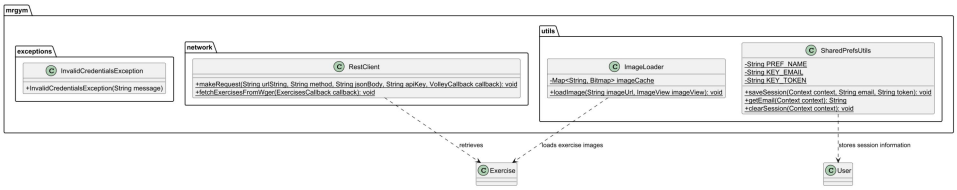


Figure 30

3.7 User Interface

The last part of the system architecture is the user interface. This includes a rough sketch, as well as how it looks when the application is running. Our user interface is mainly composed of the illustrations present in Figure 31, Figure 32, Figure 33, Figure 34, Figure 35, Figure 36, Figure 37 and Figure 38.

Layout: Login

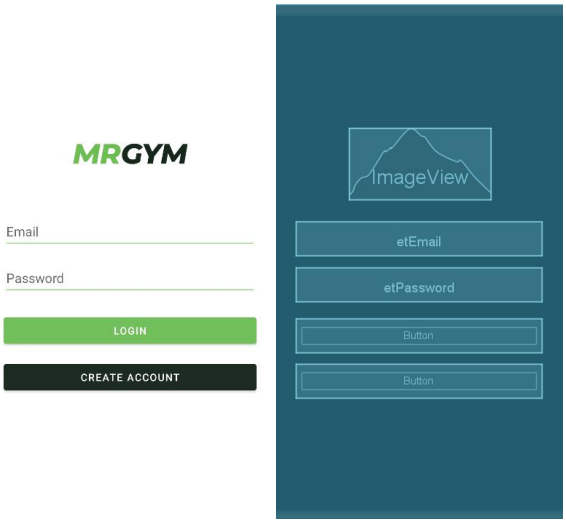


Figure 31

Layout: Register

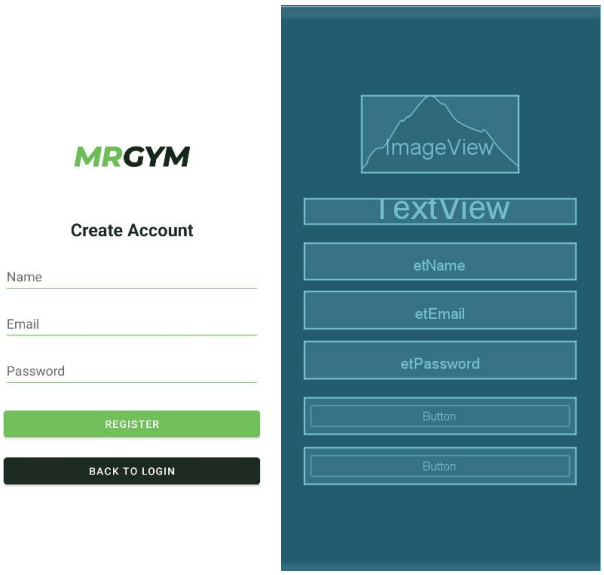


Figure 32

Layout: Profile

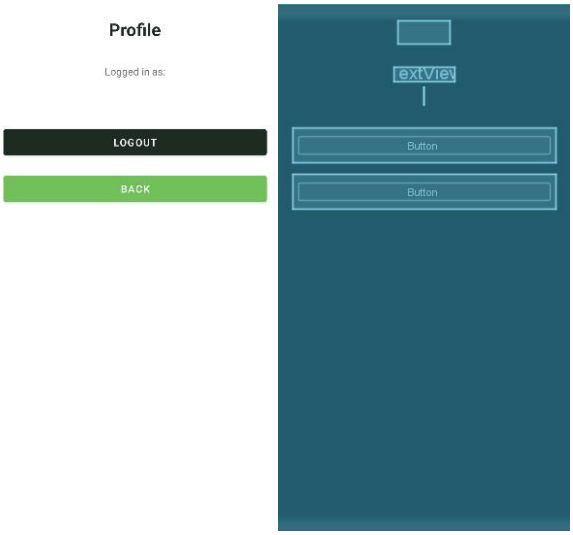


Figure 33

Layout: Main Activity

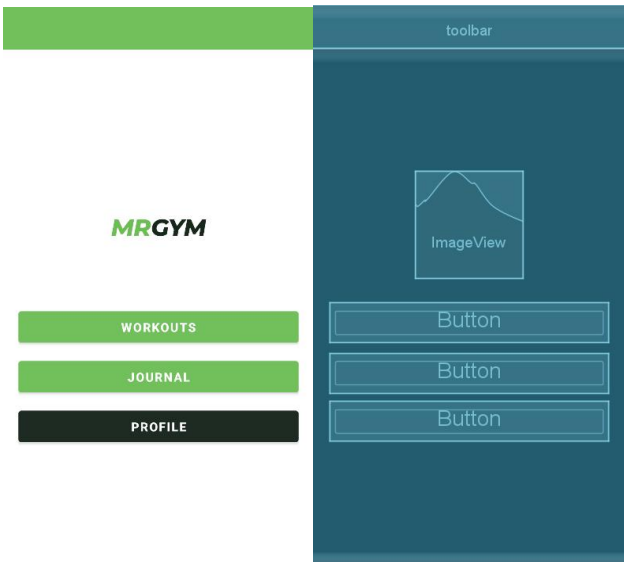


Figure 34

Layout: Workout

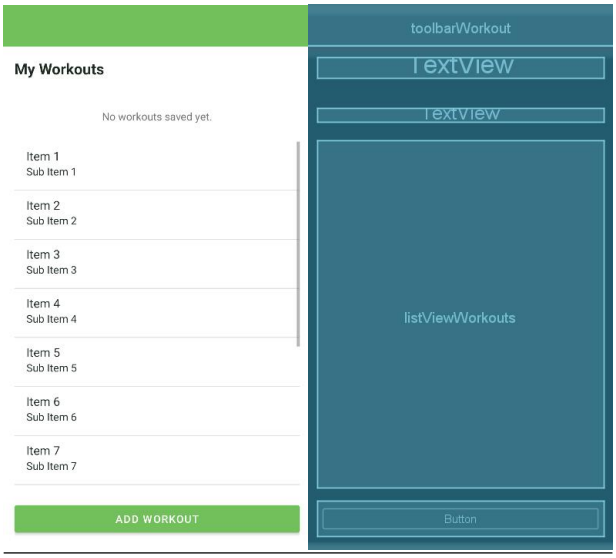


Figure 35

Layout: Create Workout

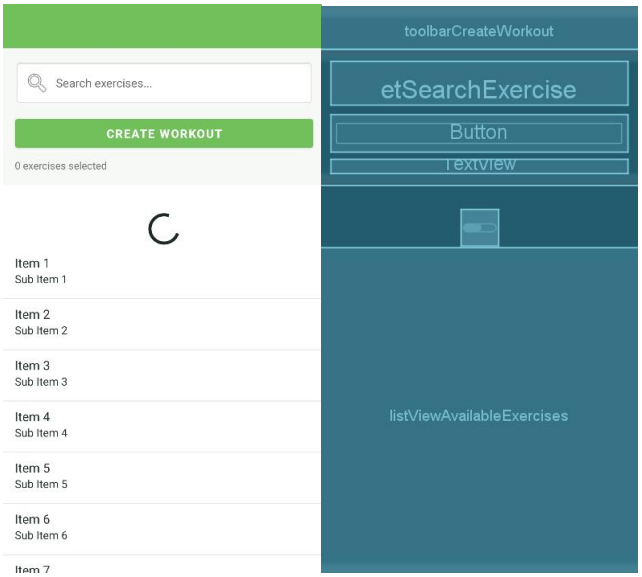


Figure 36

Layout: Journal

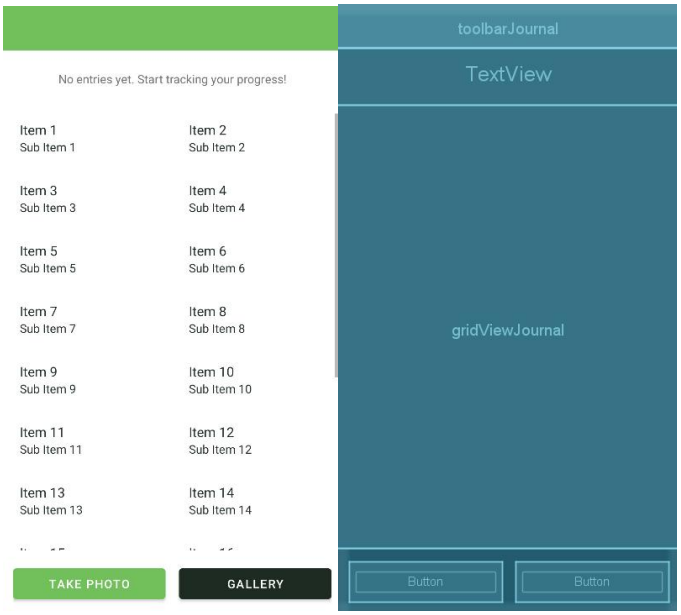


Figure 37

Layout: Journal Entry

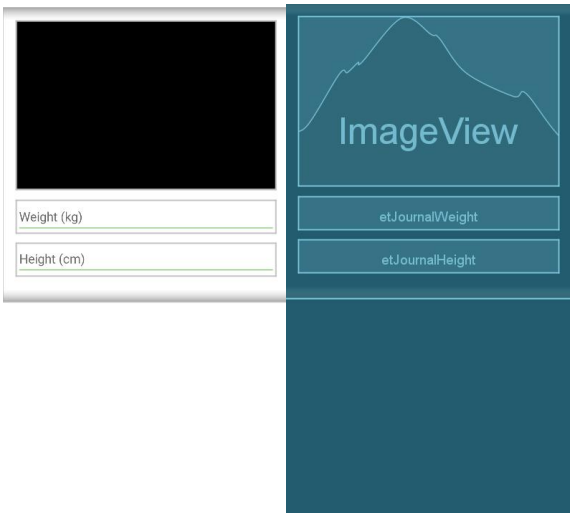


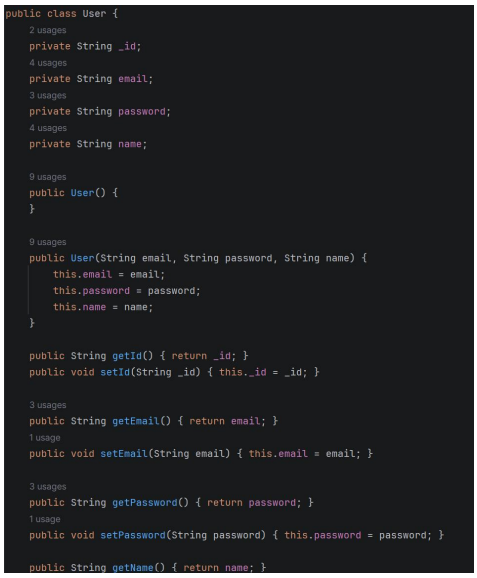
Figure 38

6 Implementation

To implement the app according to what was proposed in the previous steps, and due to what was proposed by our teachers, we used the IntelliJ IDEA and programmed the code in the Java language.

5.1 Code

In Figure 39 there's an example of an existing model class "User". As with all classes, we adapted the GRASP Information Expert Principle, meaning in this example that the class user is responsible for all data of a user, and is also responsible for their management.



```
public class User {
    2 usages
    private String _id;
    4 usages
    private String email;
    3 usages
    private String password;
    4 usages
    private String name;

    9 usages
    public User() {
    }

    9 usages
    public User(String email, String password, String name) {
        this.email = email;
        this.password = password;
        this.name = name;
    }

    public String getId() { return _id; }
    public void setId(String _id) { this._id = _id; }

    3 usages
    public String getEmail() { return email; }
    1 usage
    public void setEmail(String email) { this.email = email; }

    3 usages
    public String getPassword() { return password; }
    1 usage
    public void setPassword(String password) { this.password = password; }

    public String getName() { return name; }
```

Figure 39

Figure 40 shows how we fetch exercises from Wger. By using the helper method makeRequest(), we use GET to retrieve information from the Wger API. When it succeeds, we get the exercise ID, its image record, and the URL to where the image is stored.



```
1 usage
public static void fetchExercisesFromWger(final ExercisesCallback callback) {
    String imgUrl = "https://wger.de/api/v2/exerciseimage/?limit=400";
    makeRequest(imgUrl, method: "GET", jsonBody: null, apiKey: null, new VolleyCallback() {
        1 usage
        @Override
        public void onSuccess(String imgResult) {
            final Map<Integer, String> imageMap = new HashMap<>();
            try {
                JSONObject imgRoot = new JSONObject(imgResult);
                JSONArray imgArray = imgRoot.getJSONArray( name: "results");
                for (int i = 0; i < imgArray.length(); i++) {
                    JSONObject obj = imgArray.getJSONObject(i);
                    int exId = obj.getInt( name: "exercise");
                    String img = obj.optString( name: "image", fallback: "");
                    if (!img.isEmpty() && !imageMap.containsKey(exId)) {
                        imageMap.put(exId, img);
                    }
                }
            } catch (Exception ignored) {
            }
            loadTranslations(imageMap, callback);
        }

        1 usage
        @Override
        public void onError(String error) { loadTranslations(new HashMap<Integer, String>(), callback); }
    });
}
```

Figure 40

Figure 41 shows the code for the profile activity. Android uses an event-driven system, where user interactions trigger specific actions. In this case, btnLogouProfile and btnBackProfile each have an OnClickListener attached to them, one refferencing the “Logout” button and the other refferencing the “Back” button in Figure 33. After getting the current user email (if it is not available, it automatically sets it to *guest@mrgym.com*), each button gets its own properties for what it will do.

```
private Button btnLogoutProfile, btnBackProfile;

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_profile);

    tvUserEmail = findViewById(R.id.tvUserEmail);
    btnLogoutProfile = findViewById(R.id.btnLogoutProfile);
    btnBackProfile = findViewById(R.id.btnBackProfile);

    String email = SharedPrefsUtils.getEmail( context: this);
    if (email != null) {
        tvUserEmail.setText(email);
    } else {
        tvUserEmail.setText("guest@mrgym.com");
    }

    btnLogoutProfile.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View v) {
            SharedPrefsUtils.clearSession( context: ProfileActivity.this);
            Intent intent = new Intent( packageContext: ProfileActivity.this, LoginActivity.class);
            intent.setFlags(Intent.FLAG_ACTIVITY_NEW_TASK | Intent.FLAG_ACTIVITY_CLEAR_TASK);
            startActivity(intent);
            finish();
        }
    });

    btnBackProfile.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View v) { finish(); }
    });
}
}
```

Figure 41

5.2 Tests

Tests were also made, to make sure of the well-functioning of the application. Our test results and code are as follows in Figure 42, Figure 43 and Table 2:

```java
package mrgym.models;

import ...

public class UserTest {

    @Test
    public void testDefaultConstructor() {...}

    @Test
    public void testParameterizedConstructor() {...}

    @Test
    public void testSettersAndGetters() {
        User user = new User();
        user.setId("user_123");
        user.setEmail("athlete@test.com");
        user.setPassword("secret456");
        user.setName("Athlete");

        assertEquals( expected: "user_123", user.getId());
        assertEquals( expected: "athlete@test.com", user.getEmail());
        assertEquals( expected: "secret456", user.getPassword());
        assertEquals( expected: "Athlete", user.getName());
    }

    @Test
    public void testToString() {
        User user = new User( email: "john@test.com",  password: "123456",  name: "John Doe");
        String result = user.toString();
        assertTrue(result.contains("John Doe"));
        assertTrue(result.contains("john@test.com"));
    }
}
```

Figure 42 - User Tests



Figure 43 - User Tests Results

Table 2 - Overall Test Results

Test Type	Total Tests	Passed Tests
Exercise	8	8
JournalEntry	7	7
User	4	4
WorkoutLog	5	5
WorkoutPlan	6	6

As seen in Table 2, the test success rate was 100%.

6 Conclusion

After spending weeks on the MRGym project, we learned about Java and Android development, and feel as if we accomplished what we first expected to do with our

project proposal. Although our work had setbacks at times, we managed to complete a fully functional application that we feel has potential to help us not only as programming students, but also in training and the fitness world.

A possible update for the application could be an outside tracker, which would help the user with walking and running by tracking their time and space covered, as well as more relevant data through the GPS hardware. We initially had this idea, however ended up focusing on having everything fully functional.

We hope to have clarified everything in the best manners through this report and tried to exploit every part of the development of the application as clearly as possible.